



Requirements I — Why Users Cannot Tell You, and Who to Ask

The most expensive defect there is, the four reasons nobody can simply state what they need, and how to find the stakeholder who is not in the room

Session 7 of 28 · Tuesday 29 September 2026 · 1:00 – 2:20 PM

· Prof. Guillaume Sanje Mpacko, Ph.D.

SESSION 7 OF 28

Requirements I — Why Users Cannot Tell You, and Who to Ask

Tuesday, 29 September 2026 · 1:00 – 2:20 PM · Unit 2 · Requirements

LEARNING OBJECTIVES

- ✓ Make a team decision with its trade-offs written down, and turn a logged task into a plan: the check, the steps, and what it waits on.
- ✓ Name the four reasons a stakeholder cannot simply tell you what they need, and the countermeasure for each.
- ✓ Find the stakeholders of a system — including the non-users and the empty chair — and place them on a power-interest grid.
- ✓ Order the elicitation instruments so that each one covers the blind spot of the one before.

AFTER CLASS — BEFORE FRIDAY

Read the **corrected progression logs** for Sessions 5 and 6 in the team repository (</project/progression/>), especially the 📁 boxes, and Decision 2 in Part 1 below — the five technologies and the three stacks. Friday's block opens with that decision.

PART 1 · TEAM PROJECT WORK BLOCK

first 50 minutes of class

Lessons from the logs, one decision, one model plan, one task per group

Today the team catches up before it speeds up: what the first two logs teach, the paper-register decision, a tour of the stack options — decided on Friday, once everyone has had time to read them — and one real plan worked in full before each group writes its own.

THE FIFTY MINUTES

10 · lessons from the logs	3	8 · stack tour	10 · model plan	13 · groups: one task each	3 A	3 B
corrected S05 and S06 logs	Decision 1	decided on Friday	S05 #3a, walked through	tasks that wait on nothing	present	

LESSONS FROM THE FIRST TWO LOGS — THE CORRECTED VERSIONS, IN TEN MINUTES

The corrected `S05_progression.md` and `S06_progression.md` are in </project/progression/>. The instructor walks through five lessons; the 📁 boxes in the files give the full reasoning. Read them after class.

#	Lesson	What the log said	What it should have said
1	Nothing had been built — and that error was the lecture’s, not yours. Your instinct to turn S-01 to S-03 into work was right.	“Sprint 1”; review what was “built in sprint 0”	No sprint yet. Scrum adopted on 22 Sept; sprint 1 runs 5–16 Oct
2	A dependency points from what is built into something, to the thing it goes into.	“S-01 on hold until S-03 is done”	The S-03 import waits for S-01 (the store) and S-02 (identifiers) — not the reverse
3	Evidence is a check somebody else could carry out.	“All data moved successfully without errors”	Patient count per village matches; 50 rows compared field by field, 0 differences
4	Every date falls before the review, or the sprint has failed on paper.	Due 18–19 October	Done by Wed 14 Oct; dry run Thu 15; review Fri 16
5	Write down what you decided, predicted and waited on — the same day.	No answers, no P1–P3, no “waits on”; committed 6 days late	Answers recorded; a “Waits on” column; committed the day of the session

DECISION 1 · THE PAPER REGISTER — MADE BY THE INSTRUCTOR (ADR-002)

Decision: SANTE-BASSAM replaces the **consultation register** (option A). The team does not vote on this one: it is decided for you, to give the block’s time to the stack. The reasoning is below, because a decision nobody can explain is not engineering.

A · Consultation register — chosen

- + Covers **every visit** — the whole post and its 6,000 people, as Session 2 scoped it.
- + Fits the backlog **as written**: the nurse’s notebook (S-12), visits by village (S-13).
- + Its hardest part — duplicates, no card, a rule nobody wrote down — is exactly what Unit 2 teaches you to handle.
- + A wrong result is an inconvenience you can see and correct.
- The follow-up rule is tacit: you must draw it out of the nurse.
- Card numbers are not unique; finding patients is harder.

B · Child immunisation register — not chosen

- + Explicit rules: every next dose has a due date — wonderful to specify precisely.
- + Searching by the mother’s name is natural; tracing missed doses has high public-health value.
- Vaccine schedule rules (intervals, minimum ages, catch-up doses) need clinical expertise the team cannot check. **A wrong due date can harm a child.**
- Every record is a minor’s: guardian consent (DR-02) on everything.
- Covers only children under five, so the post’s main register would stay on paper.

How the swing went — four criteria, scored 1 to 3

	A · consultation
Clear, precise rules	1
Fits scope and backlog	3
Safe for first-time builders	3
Matches what Unit 2 teaches	3

	B · immunisation
	3
	1
	1
	2

A: 10

B: 7

B wins on precision;
A wins on safety and fit.

The deciding argument: B’s hardest part is clinical — a schedule the team cannot verify, where an error harms a child. A’s hardest part is engineering — finding patients and drawing out a tacit rule — which is what this course teaches. **Revisit if** the nurse saying the immunisation register is where the post loses the most time.

DECISION 2 · THE TECHNOLOGY STACK — PRESENTED TODAY, DECIDED ON FRIDAY (ADR-003)

Today: the instructor tours the five technologies and the three stacks below, and answers questions. **Before Friday:** each of you scores the three stacks **alone** (HW07 Q4). **Friday:** the six scores go on the board, the team argues about the differences, and decides. Scoring alone first stops the loudest voice deciding for everyone — a habit of good engineering teams.

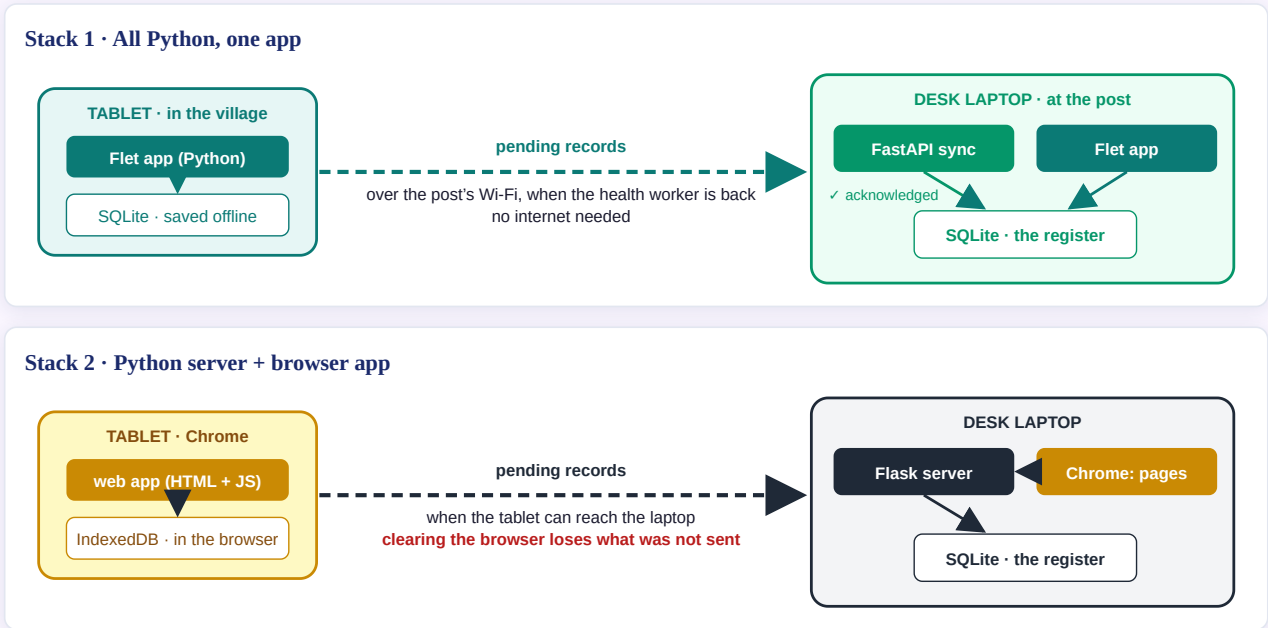
THE FIVE TECHNOLOGIES BEHIND THE THREE STACKS

Flet <i>apps with a real user interface, in Python</i> RUNS ON laptop Android tablet browser GIVES YOU buttons forms lists one codebase WATCH OUT Newer; its API has changed between versions — pin one. The first Android build takes setting up.	FastAPI <i>a data service other programs call</i> RUNS ON desk laptop GIVES YOU JSON in and out a free /docs test page input checking WATCH OUT No screens of its own: it is the receiver for sync, not the app people use.	Flask <i>a small, classic Python web server</i> RUNS ON desk laptop GIVES YOU web pages JSON huge community WATCH OUT Pages need HTML, and working offline in the browser needs JavaScript.	Streamlit <i>a data app from one Python script</i> RUNS ON desk laptop browser GIVES YOU tables charts quick forms reports WATCH OUT Re-runs the whole script on every click: fine for reports, awkward for long forms. Not for tablets offline.	KoboCollect <i>a ready-made offline form app</i> RUNS ON Android tablet GIVES YOU offline forms forms written in a spreadsheet upload when online WATCH OUT Data goes to Kobo's server; forms, not an app — searching the whole register is limited.
--	---	---	--	---

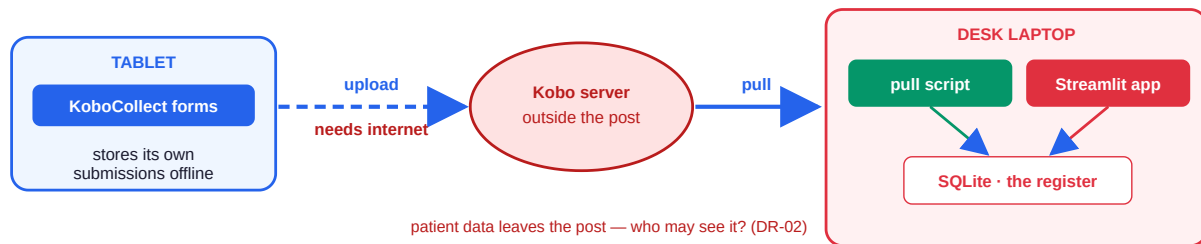
What the team needs	Flet	FastAPI	Flask	Streamlit	KoboCollect
Screens on the desk laptop	yes	no	yes	yes	no
An app on the Android tablet	yes	no	as a web app	no	yes
Saves offline on the tablet	SQLite	—	IndexedDB, via JavaScript	no	built in
Receives synced records	no	yes	yes	no	Kobo's server
Team writes only Python	yes	yes	+ HTML, JS	yes	+ a spreadsheet
Easy for a first-timer	medium	medium	medium	easy	easy
How well frontier models know it	fair	very well	very well	very well	fair

No single technology does everything, which is why every stack combines two or three. Green: does it well. Yellow: with effort or help. Red: does not do it.

THE THREE STACKS — WHERE A PATIENT RECORD TRAVELS



Stack 3 · Python on the desk + a ready-made form app



Read the arrows before the boxes. Stack 1 and 2 keep every record inside the post; stack 3 sends it through a server outside. Stack 1 is Python end to end; stack 2 needs JavaScript on the tablet; stack 3 needs almost no code on the tablet but internet to move data.

The six questions you will score each stack on, alone, before Friday (1 to 3 each):

Six questions, each scored **1 to 3 for each stack**. “The six” means the six of you. Score a stack by its **weakest essential part**: a stack that is all Python except the one piece that saves data offline is not “all Python”.

Question	What it really asks — and why it matters here	3	2	1
1 Can every one of us review all of the code?	Rule 2 says nothing generated merges until a teammate who did not prompt it has read it line by line. If part of the code is in a language only one or two of you read (JavaScript, say), only they can review that part: every change waits for them, and when they are absent nobody can fix it. It is a question about the team, not the technology.	All of the team’s code is Python.	Python, plus a small, separate piece in something else — e.g. a form written in a spreadsheet.	An essential part — e.g. the tablet’s offline saving — is in a language most of you do not read.
2 Can the tablet do its whole job with no network, all day?	On outreach days the health worker takes the tablet to villages with no Wi-Fi and no mobile data , for eight hours. There it must register new patients, find existing ones (to avoid duplicates) and keep every record safe until it is back at the post to sync. A record lost in a village is a patient’s history lost.	Registers, searches and stores everything on the tablet itself; nothing is lost if the battery dies.	Registers offline, but searching existing patients is limited, <i>or</i> an ordinary action (clearing the browser) can lose unsent records.	A core function needs a connection — to the laptop or to the internet.
3 How much new tooling stands between us and the first saved record?	Sprint 1 ends on 16 October. Count what you must install and learn before one patient is saved on the tablet: a new build system, a new language, an account on an outside service. Each one is days of set-up nobody sees in the review.	Python and one or two libraries; first record within a few days.	One new toolchain (an Android build, or browser offline code); about a week.	Several new pieces, or one whose set-up nobody can predict.
4 Where does patient data live, and who else could see it?	These are health records (DR-02). Follow the arrows in the three diagrams: every place a record sits or passes through is a place it can leak or be lost. Data that never leaves the post’s own devices can be backed up by copying one file.	Only on devices at the post, in files the team controls and can copy.	At the post, but partly in browser storage the team cannot easily inspect or back up.	Passes through a server outside the post, run by someone else.
5 How well do frontier AI models know every part of this stack?	The AI will write most of the code, from your descriptions. It writes good code for tools with many years of public examples, and out-of-date code for tools whose interface changed recently — code that looks right and fails. Look at the “how well models know it” row of the capability table.	Every part is mainstream and stable (e.g. Flask, SQLite, FastAPI).	One part is popular but young or fast-changing — pin its version and check.	An essential part is niche; examples are scarce.
6 When it breaks in February and the team has gone, can the post recover?	After December nobody from the team is there. The clerk and the nurse have a one-page sheet. The fewer moving parts, accounts and passwords, the more likely a restart or a copied file fixes it.	One app and one database file: restart, or restore by copying a file.	Two parts that must both run (an app and a sync service; a browser and a server).	Depends on an outside service, account or password someone must keep alive.

The six are not equally important. Friday starts with one question: **for a rural health post, which rows matter most?**

Every team decision is written down as a decision record — one page in /arch/

1. **Context** — what forced the decision now.
2. **Options** — at least two, each with what it costs and what it buys.
3. **Decision** — which one, in one sentence.
4. **Who decided, and who disagreed** — a team that records its disagreements can revisit them honestly.
5. **Revisit if...** — the event that would reopen it (for example, “the Android build fails the one-day trial”).

EVERY PLAN HAS SIX PARTS

1 The check

How we will know it is done: Given / When / Then, or what the stand-in clerk will see. It is the heart of what you give the AI.

2 Design choices

What you chose, what you rejected, one line why.

3 Steps

Numbered. Each a verb and an object, half a day or less.

4 Waits on / blocks

Including anything owned by the other group.

5 AI use

What you give the AI; which teammate reads the code before it merges.

6 Open question

For the Product Owner or the instructor — or “none”.

THE MODEL PLAN — A REAL TASK, OWNED, WAITING ON NOTHING: S05 #3A, THE PATIENT IDENTIFIER FORMAT

Three actions in the logs have an owner and wait on nothing: S05 #1 field map (AKAHI), S05 #3a identifier format (TRAORE) and S06 #7a search-screen prototype (TOURE). **#3a is the most complex of the three**, for three reasons:

- it can never be undone: once 6,000 paper cards carry a number, the format is frozen;
- four other pieces of work wait on it;
- it must work on two devices that cannot talk to each other while the tablet is in a village.

The instructor walks through it in class; the other two go to the groups. Owner **TRAORE**, due **Thursday 1 October**. Each part gives the answer, then *why* it is written that way.

1 • The check

Two teammates who did not write the format, working separately with only `arch/identifier-format.md`, write the identifier for five cases. The check passes when **both write exactly these answers**:

Case	Expected identifier
(a) the laptop’s first new patient of 2026	BSM-L-26-00001
(b) the tablet’s first new patient, the same morning, offline (the post has one tablet)	BSM-T1-26-00001
(c) the laptop’s 10,000th new patient of 2026	BSM-L-26-10000
(d) the first of the 6,000 patients imported from the paper register	BSM-M-26-00001
(e) the same tablet’s first patient after its app was reinstalled	BSM-T2-26-00001

...and all five are different; each fits in the number box of the paper card; and each, read aloud by one teammate, is written down correctly by the other.

Why: this is rule 1 — the check is the precise description, and it can be run by hand today, before any code exists. Writing the expected answers down is what exposes cases (d) and (e), the two a first draft forgets: imported patients also need numbers, and a reinstalled app forgets its counter. “Both write the same thing” tests that the page is unambiguous; “read aloud” tests that it survives a busy clinic.

2 • Design choices

Chosen: `POST-DEVICE-YEAR-SEQUENCE`, 15 characters at most.

- **BSM** — the post. *Why:* if a second post ever uses the software, numbers stay unique across posts.
- **L, T1, M** — where the number was made: **L** the laptop; **T1** the post’s one tablet, first installation of the app; **M** the import — patients **m**igrated from the paper register (not **I** for “imported”: the rule below excludes I, because handwritten I reads as 1). The code names an **installation of the app**, not a piece of hardware. The post has one tablet; if its app is ever reinstalled, or the tablet replaced, the new installation is

T2 — still one tablet. *Why:* each installation counts on its own, so the laptop and the tablet working offline **can never produce the same number** and never need to talk to agree.

- **26** — the year. *Why:* the counter restarts each January, which keeps the sequence to five digits; and the clerk can see when a patient was registered.
- **00001–99999** — the sequence, per device and per year. *Why:* the post has about 6,000 patients in all; 99,999 a year per device leaves a huge margin.
- **Rules:** capitals and digits only, never the letters O or I; hyphens printed and stored, but search accepts the number typed without them; an identifier never changes and is never reused; a reinstalled app or a replaced tablet takes the next code (T1 → T2 → T3), and codes are never reused. *Why:* handwritten O/0 and I/1 are confused; the clerk will not always type hyphens; a reinstalled app restarts its counter at 1 and would re-issue numbers already on cards.

Rejected:

- A random code (UUID) — unique with no coordination, but 36 characters: it cannot be written on a card or read aloud.
- Recovering the tablet’s last number from the laptop after a reinstall, so it can keep T1 — fails exactly when it matters: a reinstall in a village, with no network, cannot ask the laptop.
- One shared counter (1, 2, 3...) — short, but the tablet in a village cannot ask the laptop for the next number, so offline it would duplicate.
- The paper register’s line number — already exists, but it is not unique: numbering restarted with a new register book in 2023.
- A code built from name and birth date — changes when a spelling is corrected, and namesakes born the same year collide.

Deferred: a check digit (one extra character that catches most copying mistakes). **Revisit if** the sprint 1 review shows numbers being mistyped; until then, search by name catches a mistyped number.

Why: every choice carries its reason, and every rejected option is named with the reason it lost. In March, when someone asks “why not just 1, 2, 3?”, the answer is already written and the team does not argue it again.

3 • Steps

1. **Measure** the number box on the paper card and photograph it. *Why:* the length limit is a fact to look up, not an opinion.
2. **List** every source of new patients — the laptop, the one tablet, the import — give each a code, and write the rule for a reinstalled app or replaced tablet (T1 → T2). *Why:* a forgotten source is a future collision.
3. **Write** `arch/identifier-format.md` on one page: the pattern, what each part means, the five cases of the check, what happens at New Year and when a device is replaced, and what the number is *not* (not the national health ID, not the register line number).
4. **Give** the page to the AI as a reviewer (part 5), and answer each hole it finds in the page — or write why it is not a hole.
5. **Run** the check: two teammates, separately. Wherever their answers differ from the table, the page is ambiguous: fix the page, not the teammates.
6. **Send** to the instructor by Thursday; fix; mark it version 1.0 and commit. From then on it changes only through a new decision record.

Why: each step is a verb and an object, and small enough to finish in one sitting; together about one day. Step 1 comes first because its answer can invalidate everything after it.

4 • Waits on / blocks

Waits on nothing: it is a one-page description; it needs no stack, no data and no other task. **Blocks four things:** S05 #3b, the code of `new_patient_id()`; S05 #4, the import (each imported patient needs an M number); S06 #2, the registration form (it shows the new number); and S-16, the printed card.

Why: a task that waits on nothing and blocks four others is done *first*. That is why it is due Thursday, before sprint 1 starts, while the tasks it blocks are due the week after.

5 • AI use

The AI is a **reviewer** here, not the author. The prompt, in full: “*Here is our patient identifier format [paste the page]. A laptop and a tablet both create patients while offline and do not talk to each other until evening. List every situation in which two patients could get the same identifier, or one patient two identifiers. For each, say*

which rule in the page fails.”

What a good review finds, and the answer for each:

- **The app is reinstalled and its counter restarts at 1** → the page already says the new installation takes the next code, T2.
- **The tablet’s clock shows the wrong year** → add a rule to the page: the year comes from the device clock, so the tablet’s clock is checked against the laptop’s at every sync.
- **The same patient is registered twice, once on each device** → the format cannot prevent this. Write that in the page: duplicates are caught by searching before registering (S-05) and by reconciliation (#4).

Rule 2: MANOUAN, who did not write the page, reads the page and the AI’s list before it goes to the instructor. Later, the frozen page is exactly what the AI is given to write `new_patient_id()` in #3b.

Why: an AI is at its best finding holes in a precise description, and the third finding teaches the most: a good design says plainly what it does *not* solve, and where that problem is solved instead.

6 · Open question

- **For the instructor:** does the Ministry prescribe a patient-number format (DR-01)? *If yes:* the official number is stored as an extra field, and our format stays the post’s own number. *If no:* ours is the only number.
- **For the Product Owner** (to ask the stand-in clerk): is the number ever read aloud — to the nurse, or on the phone? *If yes:* keep the hyphens and the short device codes exactly as designed. *If no:* nothing changes, but the check’s “read aloud” clause can be dropped.

Why: each question states what changes with each answer. A question whose answer would change nothing is not worth anyone’s time.

What makes this a model: the check has expected answers anyone can compare against; every choice has a reason, and every rejected option too; the AI attacks the description instead of writing it; and each question says what it would change.

WHICH TASKS WAIT ON NOTHING TODAY?

A group can only plan well what it can start tomorrow. With the register decided and the stack not yet chosen, three actions wait on nothing. The most complex is the model plan above; the other two go one to each group.

Action	Owner	Waits on	Free today?
S05 #1 · field map of the consultation register	AKAHI	— (ADR-002 made)	Group A
S05 #3a · the identifier format	TRAORE	—	the model plan
S06 #7a · paper prototype of the search screen	TOURE	—	Group B
S05 #2 · synthetic-register generator	TETE	#1	not yet
S05 #3b, #5, #7 · generator code, safe save, repository	TRAORE, TOURE	the stack	after Friday
S05 #4, #6 · import, recovery	BIA, MANOUAN	#1, #2, #3b, #5	not yet
S06 #1–#6, #7b, #8 · registration and search	all	S05 actions, the stack	Friday onwards

Group A · AKAHI, TETE, BIA

plans one task, owned by AKAHI

- **S05 #1** · map every column of the consultation register and the patient card to a Patient field, and list what has no home — due Fri 2 Oct

Group B · TRAORE, TOURE, MANOUAN

plans one task, owned by TOURE

- **S06 #7a** · paper-prototype the S-05 search screen and run it with the stand-in clerk — due Wed 7 Oct

Thirteen minutes, all six parts, to the standard of the model. The owner writes; the other two challenge. Neither task depends on the stack, so neither waits for Friday.

THE PRESENTATIONS

Three minutes per group: **the check, the design choices and the first step**. The other group asks **one** question. The Scrum Master writes the question, the answer and any change in the log.

ONE MORE EXAMPLE OF A MODEL PLAN — A STORY NOBODY HOLDS (S-16, PRINT A PATIENT CARD)

Read this one after class. S-16 is in the backlog but in no sprint yet, so nobody owns it. It is shown because it is the opposite of #3a in two ways: it is **code**, not a description; and its open question could **cancel or reshape the whole story**.

1 • The check	Given Awa KONE is registered as <code>BSM-L-26-00017</code> — village Azuretti, born 1994, mother Adjoua KONE — <i>when</i> the clerk opens her record on the desk laptop, network off, and presses “Print card”, <i>then</i> within 5 seconds a one-page PDF the size of the paper card opens, showing in the paper card’s order: post name, identifier in large type, family name, given names, sex, birth year, village, mother’s name — and nothing else. Also: pressing “Print card” again later prints the same identifier; and N’GUESSAN Adjoua Épiphanie prints with her apostrophe and accent correct. Why: “nothing else” is a privacy rule: the patient carries the card and others see it, so no diagnosis and no visit reasons. The accented name is the case that breaks PDF code most often. “The same identifier” guards against a reprint silently creating a new patient.
2 • Design choices	Chosen: make the PDF on the desk laptop with <code>fpdf2</code> — pure Python, one <code>pip install</code>, no internet, no system libraries — so it works with any of the three stacks, which all have Python on the laptop. Embed a Unicode TrueType font (DejaVu Sans), because <code>fpdf2</code>’s built-in fonts cannot print every character (the curly apostrophe ’, for one). Copy the layout of the paper card, so the clerk and patients recognise it. Rejected: reportlab (works, but far larger than a one-page card needs); an HTML page printed from the browser (the size and margins change with each browser and printer setting, so cards would come out different sizes); an online print or PDF service (needs internet, and sends patient data out of the post); a QR code on the card (nothing at the post can scan it — revisit if a scanner arrives). Why: each rejection names a concrete failure at <i>this</i> post: no internet, no scanner, a clerk who must recognise the card. Reasons tied to the setting are the ones that still hold in six months.
3 • Steps	0. Ask the open question (part 6) through the Product Owner — <i>before any other step</i>. 1. Photograph and measure the paper card; list its fields in order. 2. Write the check above into the story, with three synthetic patients: a short name, a long name, an accented name. 3. Give the AI the check, the Patient entry of the data dictionary, the field order and the constraints (<code>fpdf2</code>, the embedded font, returns the PDF as bytes, no network calls). It writes <code>render_card(patient)</code>. 4. Review: a teammate who did not prompt it reads it line by line (rule 2). 5. Add a “Print card” button to the patient screen that calls <code>render_card</code> and opens the PDF. 6. Check, network off: print the three cards, hold each next to the paper card, compare with the check, and record the result in the log. Why: step 0 costs ten minutes and can save every other step. Step 4 is separate from step 3 because the person who prompted the code is the worst person to review it.
4 • Waits on / blocks	Waits on: S05 #3a (the frozen identifier format, printed on the card); S-04 (a registered patient, and the patient screen the button goes on); ADR-003 (the screen framework for the button — though <code>render_card</code> itself does not wait on it). Blocks: nothing. Why: it waits on three things and blocks nothing, so it belongs <i>after</i> sprint 1. That nobody holds it is correct prioritisation, not neglect.
5 • AI use	The AI writes <code>render_card</code> from the check and the constraints in step 3 — nothing else is given to it. The reviewer checks four things: no network import (<code>requests</code> , <code>urllib</code>); the fields and their order match the check; no field beyond the card’s appears; the font is embedded. Why: giving the AI the check, rather than “make a patient card”, is rule 1 at work: the more precise the description, the less there is to fix. The reviewer’s four points come straight from the check and the design choices.

6 · Open question

- **For the Product Owner, to ask the nurse:** does the post have a working printer, paper and toner? *If no:* the story is rewritten, not built — “show the card on screen in large type, so the clerk copies it by hand onto a blank card” — a much smaller story.
- **For the instructor:** are patient cards issued by the health district? *If yes:* the post may not be allowed to print its own, and the story becomes “print the district card’s fields” or is dropped.

Why: the most valuable line in a plan is often the question that could cancel it — asked at step 0, before an hour of code.

Output: arch/ADR-002-paper-register.md (from Decision 1) and two plans in S07_progression.md. The Scrum Master commits them **today**.

Before Friday: read the corrected S05 and S06 logs; score the three stacks alone (HW07 Q4); TRAORE writes the identifier format (#3a, due Thursday).

Friday’s block: Decision 2 from your six scores, then one more task per group.

PART 2 Discussion — the last 30 minutes

Three ideas are discussed in class, about ten minutes each: sections marked **In class**. Everything marked **Read** is the reading assignment — read it before Friday.

1 · The assumption Unit 1 made **Read**

Scrum assumes a Product Owner who knows what is needed and can say it. Session 6 listed “no accessible customer” as the first place agile breaks. SANTE-BASSAM is that case: the people who know are at the post, busy, and not trained to describe their work. **Unit 2 is the remedy** — techniques for finding out what people need when they cannot simply tell you.

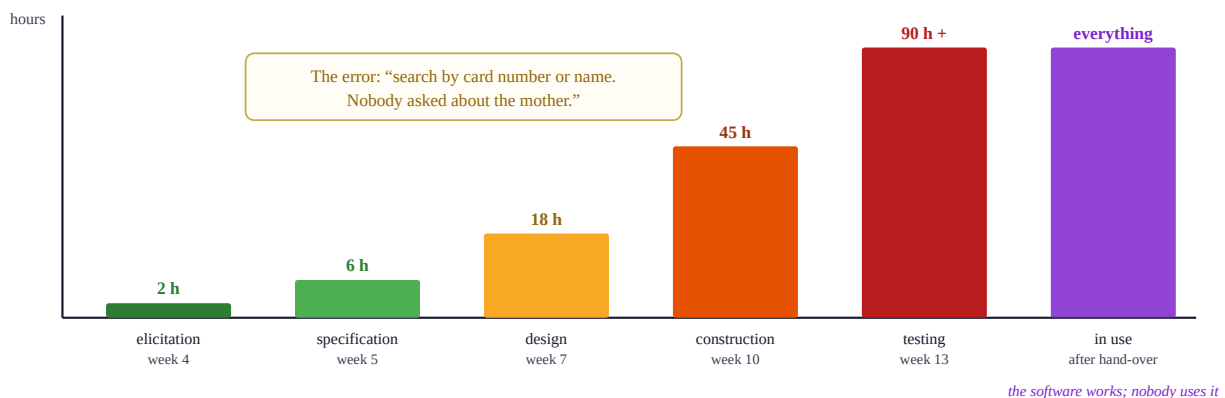


Figure 1. One requirements error, priced at each phase it survives — roughly an order of magnitude per phase (Boehm). The last bar is not a bigger number: it is working software nobody uses.

WHY THIS IS NOT A PROGRAMMING PROBLEM

Most student projects fail by building something nobody needed, *competently*. Programming skill gives no protection. Only contact with the people who will use it does.

2 · Four reasons users cannot tell you In class

The naive model is transcription: they say it, you write it down. It fails for four separate reasons, and each needs its own countermeasure — asking harder fixes none of them.

Reason	What you hear at the post	What it hides	Countermeasure
Tacit <i>they know it, cannot say it</i>	Nurse: “Who needs follow-up? You just know.”	A rule: under-fives with a second missed visit, pregnancy past 32 weeks, anyone on TB treatment.	Observe, then ask about one patient at a time — never for the rule. Write the rule down and show it to her.
Solution-first <i>a design, not a need</i>	Clerk: “I need a dropdown of the 23 villages.”	She misspells villages and the officer rejects the report. The need: <i>names the report will accept</i> .	Ask why until the answer is not about software; list two or three solutions; let observation choose.
Unarticulated <i>too obvious to mention</i>	Nothing — nobody says it.	A neighbour brings a child and the mother is absent. The screen is sometimes the only light in the room.	Read and walk before you interview. Ask about failure: “what happens when the power goes off at 7 pm?”
Conflict <i>incompatible wants</i>	Clerk: registration under 90 s. Officer: seven more fields on every record.	Neither knows, because they have never been in the same room.	Write both down with sources ; take them to someone with authority over both. The result is a requirement with a named decider .

The conflict’s ruling, from the nurse-in-charge: required fields at registration, the rest at the first visit. Written down with her name, it stops the argument recurring.

In class · diagnose it together

The district officer: “Add a column for the national ID.” Which of the four reasons is this, what is the need behind it, and what is the next question you would ask her?

3 · Who to ask: stakeholder analysis In class

DEFINITION

A **stakeholder** is anyone who **funds, approves, operates, maintains, regulates, supports, supplies data to, or is affected by** the system. Walk those eight verbs one at a time — do not try to think of people.

Then place each stakeholder by **power** (can they change or stop the project?) and **interest** (how much does the outcome affect them?). The quadrant prescribes how you engage them — that is the only reason to draw it.

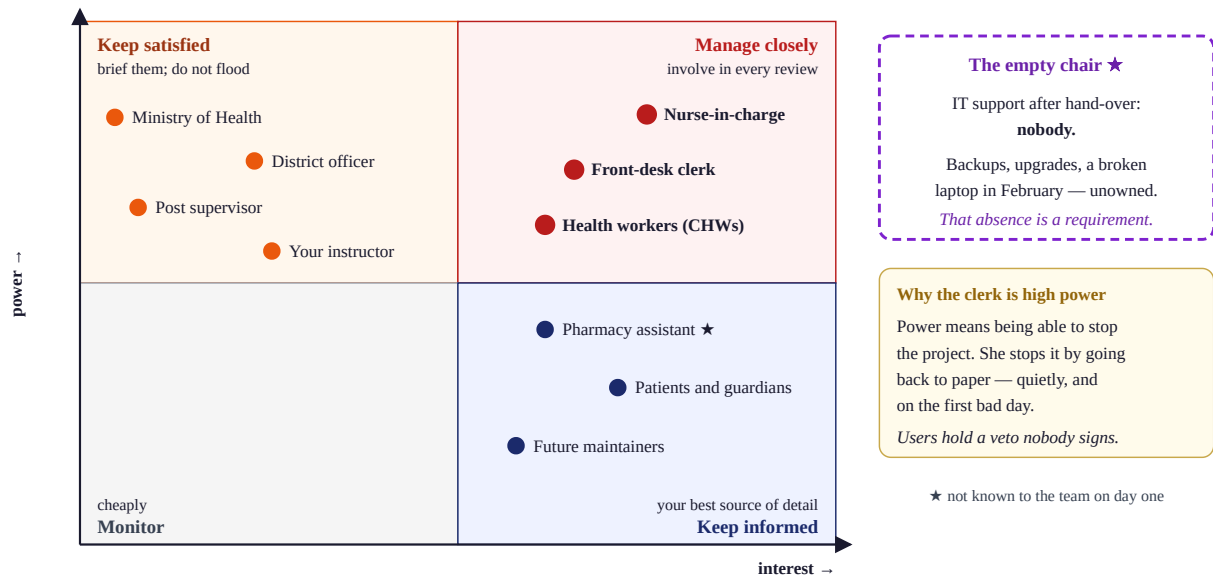


Figure 2. SANTE-BASSAM's stakeholders, placed. The pharmacy assistant was found only by asking “who else should we talk to?”. The empty chair is not on the grid at all — which is exactly why you name it.

Where the examples come from.

Throughout Unit 2 the worked examples use a **reference record set** supplied by the instructor: what a full elicitation at the post produced. Your team's own records replace it from Session 10.

In class · one minute, one verb

Who **maintains** SANTE-BASSAM after December? Name the tasks — backups, upgrades, a broken laptop — and say who does each. Where there is no name, you have found the empty chair, and a requirement.

APPLIED TO SANTE-BASSAM READ

How the six that matter most are engaged

Stakeholder	How you engage them
Nurse-in-charge	Fortnightly review; one question a day by message; signs the acceptance tests.
Front-desk clerk	Two mornings of observation; tries every increment first. Her reaction is the requirement.
Health workers	Shadow one full outreach day; they test the tablet sprint.
District officer	One 45-minute interview; a mock report before the real one.
Pharmacy assistant ★	Needs today's visit outcomes to dispense — one new story, one timing constraint.
Ministry of Health	Document analysis. It does not negotiate: identifier format, report template, retention.

4 · Which instrument, in what order In class

Every elicitation instrument is blind to something. The order below is chosen so that **each instrument covers the blind spot of the one before** — and so that you never ask what you could have read.

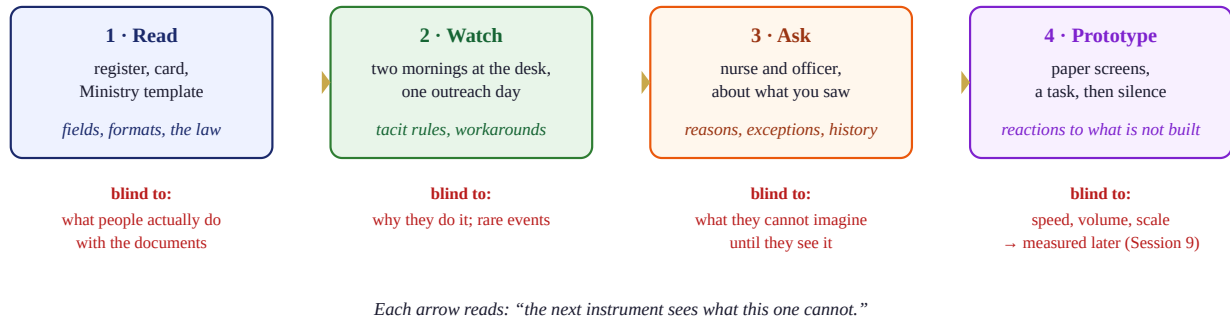


Figure 3. Read before you watch, watch before you ask. Watching first means your questions are about a notebook you have seen — which people can answer — rather than about “your workflow”, which nobody can.

Two more instruments exist. A **workshop** resolves a known conflict with the decider present — use it only if the clerk-versus-officer conflict does not settle. A **questionnaire** suits hundreds of respondents with known questions — SANTE-BASSAM has eleven stakeholders, so it is not used.

In class · back to the work block

AKAHI’s field map (S05 #1), planned an hour ago, is document analysis — step 1 of Figure 3 — applied to the consultation register. Name one question the team would have wasted on the nurse if it had asked her instead of reading that register.

SESSION 7 TAKEAWAY

A wrong requirement gets more expensive every week you do not find it. Users cannot simply tell you what they need, for four reasons with four different countermeasures. So go and look: walk the eight verbs, place everyone by power and interest, name the empty chair — then read, watch, ask, and prototype, in that order.

5 · Homework · individual · due before Session 8 (Friday 2 October)

About thirty minutes, plus the reading. It is short on purpose: your project action from the work block is the larger job this week.

TEMPLATE

Download `CSC4305_HW07_Template.md` from `CSC_4305_SWE-Assignments`, type between the `<!-- ANSWER -->` markers, **rename** it `CSC4305_HW07_FirstName_LastName.md` and **commit** it to `/homework/S07/` in your own private repository.

Markdown only. Drafting in Word is fine — convert at word2md.com and commit the `.md`.

Q1 Diagnose four statements. Tacit, solution-first, unarticulated or conflict — and the one question or action you would use next. (a) “Just make it work like the paper register.” (b) “We need an SMS reminder system.” (c) “I can tell straight away which ones will come back.” (d) The officer says every record needs a national ID; the clerk says half her patients have none.

Q2 A stakeholder nobody named. Add one to SANTE-BASSAM who is not in Figure 2. Which verb found them, which quadrant, and what goes wrong if nobody ever speaks to them?

Q3 Document analysis, for real. Take the consultation register chosen in Part 1 (and described in Session 2 §2.1). List every field it records, and flag the three you would not have guessed, with the reason each matters to the software.

Commit it by Thursday evening: AKAHI's field map (S05 #1, due Friday) is checked against all six lists.

Q4 Score the three stacks — alone. Using Part 1 (the five technologies, the three flows), score each stack 1 to 3 on the six questions, with one line of reason per score. Do not compare with anyone before Friday: the value is in six independent views.

Then one sentence: which stack would you choose, and what would make you change your mind?

Q5 Reading. The corrected S05 and S06 progression logs, the sections of this lecture marked Read (§1 and the engagement table in §3), and Sommerville, *Software Engineering* 10th ed., §4.1–4.3.

Friday: the work block plans sprint 1 (the eight actions of the corrected Session 6 log). The discussion covers the craft of asking, watching and prototyping — exactly what BIA and TOURE need for the two paper prototypes they plan in that block.

Key terms

requirements elicitation

cost-of-correction curve

tacit knowledge

solution-first framing

unarticulated need

named decider

stakeholder

the eight verbs

power-interest grid

the empty chair

document analysis

contextual inquiry

Readings and references

- Sommerville, I. *Software Engineering*, 10th ed. — Chapter 4, §4.1–4.5.
- Boehm, B. *Software Engineering Economics*, 1981 — the cost-of-change data.
- Mendelow, A. “Environmental Scanning: the Impact of the Stakeholder Concept”, ICIS, 1981 — the power-interest grid.
- Gottesdiener, E. *Requirements by Collaboration*, 2002.